

Programming The Mizar32 Board IN LISP PROGRAMMING LANGUAGE

RAMAN GOPALAN

This article aims to revive Lisp programming language for native, interactive and incremental microcontroller (MCU) program development by running a dialect of Lisp (PicoLisp) as virtual machine on the target. It also demonstrates the power of Lisp through execution of expressive Lisp programs on a 32-bit MCU.

Lisp offers a practical mathematical notation to write computer programs, mostly influenced by lambda calculus. It is still the most-favoured programming language for artificial intelligence research.

Dynamic languages like Lisp have been in existence as a versatile tool for rapid application development. Many interesting, practical embedded solutions have been developed so far with such languages, supported as part of a virtual machine (VM). Fig. 1 shows the system architecture of a natively-programmable, digitally-controlled system.

With the above architecture, it is possible to write abstract, self-adapting, middle-level drivers for hardware modules on the MCU. This enables the possibility of platform-independent, native embedded software development.

Lisp is the second-oldest high-level programming language (the first being FORTRAN). Linked lists are one of its major data structures. A program written in Lisp is constructed with lists. One of the most interesting properties of this language is its homo-iconic nature.

Although projects like PICOBIT and ARMPIT Scheme already implement a compact Lisp programming

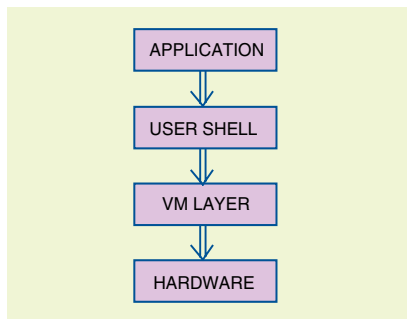


Fig. 1: General MCU software system with a VM layer

language for an MCU, there are many reasons to consider PicoLisp for programming MCUs.

PicoLisp is constructed as a VM. It is written in portable C and is easily extendable. After much research and programming to narrow down on a Lisp implementation, PicoLisp was chosen as a VM for the following reasons:

1. Dynamic data types and structures
2. Formally homo-iconic
3. Functional programming paradigm
4. An interactive REPL(read-eval-print loop)
5. Pilog, a declarative language with semantics of Prolog in PicoLisp
6. Small memory footprint
7. Permissive, non-copyleft free software licence

At the lowest level, PicoLisp programs are constructed from a single data structure called cell. A cell is a pair of machine words, which are traditionally called CAR and CDR in Lisp terminology. These words can represent either a numeric value (scalar) or address of another cell (pointer). All higher-level data struc-

tures are built out of these cells.

Basic data types that PicoLisp supports are numbers, symbols and lists. As a result, it is one of the fastest Lisp dialects available, since only fewer options are checked at runtime to parse a value.

PicoLisp in addition supports an integrated database system. This is a huge advantage for embedded system applications that require a convenient facility to perform data transactions.

Overall logical software structure for running full-fledged PicoLisp on the MCU is shown in Fig. 2. It shows the communication between PicoLisp VM and various other modules in code base.

The code base remains highly portable across various platforms and architectures simply by using the following key principles:

1. Code that is platform-independent is common code and should be written in portable ANSI C as much as possible. PicoLisp itself is a part of the common code section and is written in this way.
2. Code that is not generic (mostly peripheral and CPU-specific code) must still be made as portable as possible by using a common interface that must be implemented by all platforms on which PicoLisp runs. This interface is called platform interface.
3. Platforms vary greatly in capabilities. The platform interface tries to group only common attributes of different platforms.

Platform interface is declared in inc/platform.h header file from source distribution. It is a collection of various components like UART,